

JARVIS OS — Dossier de présentation

JARVIS OS

Infrastructure d'intelligence artificielle distribuée, on-premise et souveraine

Dossier de présentation — 17 juillet 2026 Franck Delmas · Architecte systèmes IA / Linux · Turbo31150

Synthèse exécutive

JARVIS OS transforme un simple Ubuntu Linux en un système d'exploitation IA autonome, 100 % local et sans coût cloud récurrent. Là où un Linux standard exécute des commandes, JARVIS orchestre : il route l'intelligence, s'auto-répare, se surveille et travaille en continu — sur du matériel grand public.

Ce qui le distingue d'un Linux de base

Dimension	Linux standard	JARVIS OS
Modèle d'exécution	commandes à la demande	orchestration continue (agents + timers + auto-déclencheurs)
Intelligence	aucune	routeur LLM unifié 0-token , cascade multi-modèles
Résilience	redémarrage manuel	auto-réparation systemd < 8 s , recovery GPU, backups horaires
Mémoire système	swappiness 60, governor schedutil	ZRAM zstd 3,07:1 , governor performance, HugePages, mitigations=off
Souveraineté	dépend des services tiers	0 € cloud, 0 token payant, RGPD natif
Observabilité	logs épars	widget bureau temps réel « Ce que le système fait »

Preuves chiffrées (mesurées le 17/07/2026)

- **99,6 %** de succès sur **4 413** appels LLM réels ; **91,8 %** du trafic reste sur M1 en local ; **< 1 %** cloud gratuit, **0 token payant**.
- Réponse interactive médiane **~1,1 s** ; débit jusqu'à **51 tok/s**.
- Interprète vocal 100 % local : **WER 10,51 %** sur le dataset officiel Google FLEURS, **20x le temps réel**, pour **0 €/an** (vs 1 440-2 500 €/an en cloud).
- Bande passante RAM **107 954 MiB/s**, ZRAM ratio **3,07:1**, débit CPU 16 threads **14 560 év/s** ; aucun GPU en zone thermique critique sous charge.
- Efficience : l'ensemble tourne sur **~1 % d'un petaFLOP** (matériel grand public).

Périmètre

Cluster **5 nœuds, 12 GPU** (~94 Go VRAM cumulée, **28 Go dédiés à l'inférence** sur M1), hub LLM local (ports 18800-18803), **961 agents indexés** (jusqu'à **1 435 composants actifs** toutes couches, cf. section 5), **169 skills-**

agents, bibliothèque-routeur de **~10 000 blocs** 0-token, et un widget de supervision bureau supervisé par systemd. Les nœuds HP-A/HP-B (Vocal/LLM HQ) ne sont pas toujours en ligne ; le socle permanent reste M1 + M2 + M5.

Note de rigueur. Ce dossier croise trois décomptes d'agents concordants et vérifiables (961 index cowork · 1 354 comptage entreprise · 1 435 inventaire OMEGA) — trois périmètres emboîtés, pas des chiffres contradictoires ; le détail complet est en section 5. Le « 928 » (slogan marketing `jarvis-linux`) et le badge « 900+ » (OpenClaw) mélangent agents et capacité de parallélisme : ils ne servent pas de base aux totaux vérifiés.

Sommaire

1. **Architecture & périmètre** — topologie cluster, stack logicielle, souveraineté
 2. **Benchmarks & performance** — latences, débit, local vs cloud, interprète vocal
 3. **Hardware, BIOS & couche système** — CPU/PBO, ZRAM, RAM, latence, GPU, kernel
 4. **Le widget bureau « Ce que le système fait »** — architecture & utilisation
 5. **Écosystème d'agents — le décompte réel** — trois décomptes concordants, périmètres emboîtés
 6. **Démonstration vidéo** — capture narrée du widget en fonctionnement réel
-

1. Architecture & périmètre

01 · Architecture & Périmètre — JARVIS OS

Section extraite du GitHub réel `Turbo31150/jarvis-core` et du système local M1 (2026-07-17). Tous les chiffres ci-dessous sont vérifiés depuis le dépôt et/ou la machine. Les écarts éventuels entre le slogan marketing et le réel local sont signalés explicitement.

1. Vision & positionnement

JARVIS OS — Infrastructure IA distribuée souveraine, on-premise, 0 cloud.

Auteur : **Franck Delmas · AI Systems Architect · Toulouse.**

Le système est une **couche d'orchestration multi-agents entièrement auto-hébergée** pour Linux : elle exécute des dizaines de CLIs spécialisés, des chaînes d'automatisation « Domino » et des centaines d'agents concurrents, en **remplaçant les abonnements LLM cloud (GPT-4o, etc.) à coût cloud nul**. Construite sur Docker Swarm + Redis + PostgreSQL + LLM locaux.

Positionnement en une phrase (dépôt `jarvis-linux`) : > « *The OS that never sleeps, never forgets, never bills you.* »

Trois dépôts portent le même produit sous trois angles :

Dépôt	Angle	Statut
<code>jarvis-core</code>	Cœur technique — orchestrateur, MCP, agents, watchdogs	Privé (NDA)
<code>jarvis-linux</code>	Vitrine performance / benchmarks vs cloud	Public
<code>alkymia-os</code> (AlkymIA-OS)	Offre commerciale grand public / PME, pilotable à la voix	Public — alkymia-oss.netlify.app

Dépôts complémentaires de l'écosystème : `jarvis-cowork` (workspace agents/skills OpenClaw), `jarvis-mcp-toolkit` (handlers MCP), `jarvis-browser-mcp` (automation navigateur CDP), `jarvis-zero-token`, `omertaflow` (Whisper Flow STT/TTS), `jarvis-profile` (dashboard GPU).

2. Topologie du cluster (5 nœuds)

Source : README `jarvis-core` (section « Cluster — 5 Nœuds »).

Nœud	IP	Rôle	GPU	VRAM
M1 — La Créatrice	192.168.1.85	LB1 · LM Studio · Docker Swarm	RTX 2060 + 3× GTX 1660S + RTX 3080	40 G
M2 — Le Géant	192.168.1.26	LB2 · Reasoning	3× Quadro RTX 4000	24 G
M5 — OMEGA	192.168.1.113	Relay · Vocal · Claude Code	GTX 1660S + GTX 1050 Ti	10 G
HP-A	192.168.1.90	LLM HQ	Quadro P4000	8 G
HP-B	192.168.1.91	Vocal dédié	2× GPU 6 G	12 G

Total matériel : 5 nœuds · 12 GPU · ~94 G VRAM cumulée (somme des lignes = 40+24+10+8+12 = 94 G ;

M1 = 5 GPU = 12+6+6+6+10 = 40 G).

△ **Écart de chiffre à noter** : la description courte du dépôt annonce « 68G VRAM » alors que le tableau détaillé du README et les badges affichent **88 GB VRAM**. Le total réel des lignes du tableau — avec M1 correctement compté à **5 GPU / 40 G** (les 3× GTX 1660S sont confirmées par les 28 Go dédiés à l'inférence : 3×6+10) — donne **94 G**. Retenir **94 G** (détail machine à jour) comme chiffre de référence ; 88 G reprenait un M1 à 4 GPU / 34 G, et 68 G est une valeur historique de la description, tous deux à corriger.

△ **Total GPU** : la somme des lignes donne **12 GPU** (M1 5 + M2 3 + M5 2 + HP-A 1 + HP-B 2). Les nœuds **HP-A/HP-B** ne sont pas toujours en ligne ; le socle permanent M1 + M2 + M5 représente **10 GPU** actifs en continu.

Fiches machines détaillées : `MACHINES_SPEC.md` , `docs/JARVIS_OS_ARCHITECTURE.md` , `docs/cluster/` .

3. Stack logicielle

3.1 Hub LLM 0-token & routeur

- **Load balancing double étage** : LB1 (M1) + LB2 (M2), Nginx, **failover automatique**, latence annoncée < 3 s.
- Sur M1 en local, le **hub LLM unifié** expose un proxy OpenAI-compatible : ports **18800** (hub / chat_proxy), **18801** (dashboard), **18802** (ccr — claude-code-router), **18803** (sql-bridge) — cf. `docs/PORTS_REGISTRY.md` .
- **Moteurs d'inférence** : `llama-server` , Ollama, LM Studio (M1 :1234), catalogue modèles :8500.
- **Modèles clés** : Qwen3.5-27B/9B *Opus-Distilled*, DeepSeek-R1, Gemma3, plus embeddings locaux (nomic-embed) et Whisper distil-large-v3 (CUDA).
- **Protocole 0-token** documenté : `PROTOCOL_ZERO_TOKEN.md` . Preuve de performance affichée : **96,9 % de cache hit** • **319 tokens sur 233 échanges** • **0 cloud** (`docs/PROOF_JARVIS_CLAUDE_CODE.md`).

3.2 Agents

- Le README annonce **900+ agents autonomes** (badge + description) ; `jarvis-linux` chiffre à **928 agents actifs** (uptime 99,7 %).
- Réalité vérifiable côté dépôt/machine :
 - `jarvis-core/agents/` = opérateurs Python spécialisés (browsers, container, github, mail, network, sql, telegram, voice...).
 - `jarvis-core/core/` = **480 modules Python** (`jarvis_*.py` : routeurs A/B, supervisor, mesh, planner, scheduler, aiops, admission controller, etc.) → 490 entrées de dossier au total.
 - Registre Claude Code natif chargé en session M1 (agents natifs + plugins) : **129 agents**. Le strict `~/claude/agents/*.md` (sans plugins) ne compte que **58** fichiers, retenus dans le total entreprise de 1 354 (cf. section 5). Le 129 = façade Claude Code complète ; le 58 = sous-ensemble `.md` natif consolidé.
- Familles d'agents (README) : OpenClaw (900+), skills, Antigravity CLI zero-token.

Le chiffre « 900+ » est une valeur d'écosystème global (OpenClaw + variantes). Au niveau du registre Claude Code local M1, la façade chargée en session (natifs + plugins) est de **129 agents** ; le comptage entreprise n'en retient que les **58** fichiers `.md` stricts (section 5). Les trois sont mentionnés pour transparence.

3.3 Skills

- Registre local M1 (`~/claude/skills/`) : **53 skills**.
- Registre orchestrateur `orchestrator/registry.json` : **52 skills** • **127 agents** • **2 browsers** • **181 entrées** (snapshot).
- README `jarvis-core` mentionne **61 skills** ; `jarvis-cowork` **86 skills** — valeurs par périmètre de dépôt.

3.4 Bibliothèque-routeur (anti-loop 0-token)

- **Index central de blocs** (`BLOCS-INDEX.tsv`) : ~2 579 blocs indexés (≈9 661 après expansion permanente)

- mappés sur 14 sources (prompts, n8n, scripts, docs, bases SQLite, dépôts GitHub, conteneurs, modes...).
- Principe : *une intention entre → un bloc prêt sort*, routage via `bloc.sh` avant toute délibération LLM → zéro token de compute facturé.
 - Daemon **Bibliothèque Vivante Infinie** (`cli/biblio_filler.py`) : remplissage continu 0-token via LM Studio M1 :1234.

3.5 Données, vocal, contenu

Module	Contenu (README)
Data	SQLite · PostgreSQL · Redis · Pinecone · n8n (65 flows)
Vocal	Whisper distil-large-v3 CUDA · Piper TTS · PTT Alt+X (dépôt omertaflow / Whisper Flow)
Content	Mirra autopublisher · 5 réseaux (Instagram, Threads, TikTok, YouTube) + LinkedIn CDP
MCP	24 serveurs MCP (badge) · <code>jarvis-mcp-toolkit</code> = 88+ handlers

3.6 Ports & services (M1, réel)

`docs/PORTS_REGISTRY.md` (généré 2026-07-16) : **103 ports · 23 services actifs**. Hub LLM :18800, dashboard :18801, ccr :18802, sql-bridge :18803, LM Studio :1234, BrowserOS :9003.

4. Souveraineté & RGPD

- **100 % local, 0 € cloud** : aucune donnée ne quitte l’infrastructure du client ; RGPD natif (badges `jarvis-linux` et `alkymia-os`).
- Bases et secrets restent on-premise (SQLite / PostgreSQL / Redis) ; sauvegardes chiffrées vers dépôt privé, secrets exclus de tout artefact partagé.
- Modèles LLM **open-source hébergés localement** → pas de vendor lock-in, **compatible avec toutes les IA** (bascule cloud optionnelle uniquement pour économiser des jetons, jamais imposée).
- Pilotage à la voix (STT + TTS intégrés) sans API tierce.

Résultats mesurés annoncés côté clients (`alkymia-os`) : **−14 h/semaine · 0 € cloud · ×6 plus rapide** sur le traitement des commandes · **+340 % de ROI à 12 mois**.

5. Différenciateurs

Solutions cloud classiques	JARVIS OS / AlkymIA-OS
Données dans le cloud	100 % local — données chez le client
Abonnements qui gonflent	Infrastructure possédée (0 € cloud récurrent)
Verrouillage fournisseur	Compatible toutes les IA, modèles open-source
Matériel imposé	S’adapte au parc existant — du smartphone au cluster 5 nœuds
Pilotage souris/écran	Pilotable à la voix (STT + TTS)
Latence + coût par requête	Failover LB1/LB2 < 3 s · cache hit 96,9 % · 0 token facturé

Preuves techniques distinctives : - Auto-réparation < 8 s de latence (watchdogs + supervisor). - Transcription d’1 h d’audio en 48 s (Whisper CUDA), analyse PDF en 7,9 s. - Uptime 99,7 % · agents 24/7. - Architecture entièrement documentée (`AUDIT_SYSTEME_JARVIS_OS.md` , `CAHIER_DES_CHARGES_FINAL.md` , `MASTER_ORCHESTRATION.md`).

Annexe — Chiffres à réconcilier (transparence)

Métrique	Valeur description courte	Valeur README détaillé / réel local	Retenu
VRAM totale	68 G	94 G (somme tableau machines, M1 = 5 GPU / 40 G)	94 G
GPU cluster	—	12 (M1 5 + M2 3 + M5 2 + HP-A 1 + HP-B 2)	12 (10 permanents M1+M2+M5)
Agents	900+	928 (jarvis-linux) · 129 Claude Code session (natifs+plugins) · 58 <code>.md</code> natifs stricts	900+ écosystème / 129 façade / 58 <code>.md</code>
Skills	61 (core) / 86 (cowork)	53 (~/.claude) · 52 (orchestrator)	53 local M1
Modules core Python	—	480 fichiers <code>.py</code> dans <code>core/</code>	480

2. Benchmarks & performance

02 — Benchmarks & Performance

Tous les chiffres ci-dessous sont mesurés sur le matériel réel de production (M1), pas estimés. Chaque table cite sa source. Convention : (mesuré 2026-07-17) = relevé live pour ce dossier · (archive) = mesure horodatée d'un run antérieur, conservée telle quelle. Coût récurrent de toutes ces mesures : 0 € / 0 token facturé (inférence 100 % locale ou cloud gratuit).

1. Routage LLM en production — latences réelles

Le hub unifié :18800 route chaque requête vers le backend le plus adapté (cascade 5 niveaux, failover automatique). Statistiques calculées sur le journal de routage réel data/llm_cascade_log.jsonl.

Fenêtre glissante (depuis 2026-07-15, 4 413 appels) — (mesuré 2026-07-17, via monitoring/llm_stats.py)

Métrique	Valeur
Appels routés	4 413
Taux de succès	99,6 % (4 397 ok / 16 erreurs)
Tentatives moyennes par appel	0,26 (la 1 ^{re} route suffit dans la grande majorité)
Fallbacks déclenchés	894 (20,3 %)
Latence p50	10 741 ms
Latence p95	44 444 ms
Latence p99	172 146 ms
Latence min / max	231 ms / 661 847 ms

Répartition par backend servi — (mesuré 2026-07-17)

Backend servi	Part du trafic	Rôle
lmstudio-node10/qwen3.5-9b (M1 relayé)	65,6 %	🏰 local souverain, prioritaire
lmstudio-m1/qwen3.5-9b	26,2 %	🏰 local direct
ollama/gemma3:4b	6,8 %	🏠 fallback réactif (petit modèle)
ollama-cloud/gpt-oss:120b	0,9 %	🌄 gros contexte, gratuit
autres (deepseek-r1, gpt-oss:20b-cloud)	< 0,1 %	spécialisé

Lecture souveraineté : 91,8 % du trafic reste sur M1 local (65,6 % + 26,2 %), 98,6 % au total en local (+ gemma3), moins de 1 % part vers le cloud gratuit. Aucune donnée ni token vers un fournisseur payant.

Latences p50/p95 par backend (fenêtre récente, 3 000 derniers appels du 2026-07-16 — mesuré 2026-07-17)

Backend	n	p50	p95
lmstudio-node10/qwen3.5-9b	2 700	12 831 ms	35 694 ms
ollama/gemma3:4b	252	13 231 ms	49 148 ms

ollama-cloud/gpt-oss:120b	17	10 903 ms	227 547 ms
lmstudio-m1/qwen3.5-9b (direct, sans hub)	17	85 381 ms	179 991 ms

Le p50 « global » (~11–13 s) inclut les requêtes d’**analyse longue** (400+ tokens générés). Sur les requêtes **interactives courtes**, la latence chute à ~1,1 s (voir §2). Les 85 s de `lmstudio-m1` en appel *direct* sont l’artefact connu de *reasoning-runaway* de qwen3.5 ; le hub le neutralise (prefill `<think></think>`). Source : `data/llm_cascade_log.jsonl`.

2. Débit & latence par backend — même prompt, appel contrôlé

(archive — `docs/BENCH_INFRA.md` & `docs/BENCH_BACKENDS.md`)

Infra hub :18800 , requêtes typées :

Type de requête	Latence médiane	Débit moyen
Interactif (court)	1 114 ms	16,7 tok/s
Analyse (long, ~400 tok)	9 703 ms	26,5 tok/s

Comparatif multi-backend (même prompt « API REST en 60 mots », médiane) :

Backend	Localisation	Latence médiane	Débit	Coût
M1 LM Studio (qwen3.5-9b), via hub	🏠 local	~0,4 s	11–26 tok/s	0 €
Ollama local (gemma3:4b)	🏠 local	4,5 s	30 tok/s	0 €
Ollama cloud (gpt-oss:20b)	☁️ gratuit	3,5 s	51 tok/s	0 €

Meilleure latence prod (M1 via hub) : ~0,4 s · meilleur débit (cloud gratuit) : 51 tok/s. Tous à 0 € / 0 token facturé.

3. Interprète vocal 100 % local — dataset officiel FLEURS

(archive — mesuré 2026-06-30 · repo public `Turbo31150/jarvis-interpret-benchmark`) Interprète FR↔EN complet (STT → traduction → TTS) sur laptop grand public ASUS TUF F15 (i5-11400H · RTX 3050 Laptop 4 Go, ~800 €).

Métrique	Valeur	Détail
WER (Word Error Rate)	10,51 %	FLEURS <code>fr_fr</code> , 50 clips validation
RTF / vitesse	20,2× temps réel (GPU)	faster-whisper <code>small int8_float16</code>
BLEU (traduction parole fr→en)	17,2	30 clips (BLEU sous-estime, réf reformulées)
Coût marginal	0 € / 0-token	100 % local, données jamais externalisées

Coût vs cloud — 1 000 h audio/an (prix publics 2026) :

Solution	Coût annuel
Azure (traduction temps réel)	≈ 2 500 \$/an

Google STT (+ traduction)	≈ 1 440 \$/an
JARVIS / laptop local	0 € récurrent (machine amortie en < 2 mois)

Efficience petaFLOP : le benchmark a tourné sur ~1 % d'un petaFLOP (RTX 3050 ≈ 11 TFLOPS FP16 / 44 TOPS INT8). Atteindre 1 PFLOP demanderait ≈ 91 de ces laptops (≈ 73 000 €) ou ≈ 2 H100. L'avantage n'est pas le WER brut mais **coût marginal nul + confidentialité totale + vitesse + zéro quota**.

4. GPU — état thermique & VRAM (live)

(mesuré 2026-07-17, `nvidia-smi`) — 5 GPU physiques sur M1 (RTX 2060 + 3× GTX 1660S + RTX 3080).

GPU	Rôle	Temp.	VRAM util. / totale	Charge
RTX 2060 (12 Go)	🖥️ affichage (exclu des LLM)	54 °C	4 114 / 12 288 MiB	1 %
GTX 1660 SUPER #1	inférence dispo	44 °C	972 / 6 144 MiB	0 %
GTX 1660 SUPER #2	inférence LLM active	73 °C	3 231 / 6 144 MiB	100 %
GTX 1660 SUPER #3	inférence dispo	44 °C	2 096 / 6 144 MiB	0 %
RTX 3080 (10 Go)	modèle 9B chargé	70 °C	8 745 / 10 240 MiB	0 % (idle entre req.)

Instantané pendant une charge d'inférence réelle : un 1660S sature à 100 % à 73 °C, la RTX 3080 tient le modèle qwen3.5-9b en VRAM (8,7/10 Go). Aucun GPU en zone critique. La RTX 2060 est réservée à l'affichage (règle anti-crash modeset), jamais sollicitée pour les LLM.

5. Couche système M1 — CPU / RAM / disque / cache

(archive — mesuré 2026-07-16 03:53, `logs/bench_syskernel_20260716.md`, `sysbench` réel sous charge concurrente ~7-8) CPU **AMD Ryzen 7 5700X3D** (8C/16T, V-Cache L3 96 Mo) · 46 Gio RAM + zram 24 Go zstd.

Composant	Métrique	Valeur mesurée	Verdict
CPU multi (16T)	events/s prime20k	14 560 ev/s	OK (sous load concurrent)
CPU single (1T)	events/s prime20k	1 802 ev/s	Boost actif
CPU scaling	multi/single	8,08×	Normal (SMT)
RAM séquentiel (1M/8T)	débit sysbench	107 954 MiB/s (~108 GB/s)	Excellent (DDR4 2 canaux)
STREAM (dataset > L3)	read cumulé	~15,4 GB/s R · ~10,3 GB/s W	DRAM-bound au-delà du V-Cache
L3 V-Cache 96M	accès finissant en DRAM	~8,7 % des loads L1	V-Cache retient l'essentiel
ZRAM	ratio compression zstd	3,07:1 (6,36 GiB → 2,07 GiB)	Excellent, coussin anti-OOM
Disque	randread 4k QD1	8 730 IOPS · 34,1 MiB/s	Correct (latence QD1)
Disque	randwrite 4k QD1	19 200 IOPS · 74,9 MiB/s	Bon (write cache)

Micro-benchmark de calcul brut (archive — `demo_benchmark_20260619/bench_raw.txt`) : **7 239 → 91 942 ops** en **3,43 s** (facteur ~12,7× de speedup mesuré sur le run de démo).

6. JARVIS OS vs freelance — livrable réel chronométré

(archive — mesuré 2026-06-09, `benchmark_freelance/BENCHMARK_FREELANCE.md`) Cas reproductible : scraper Python « catalogue → CSV » (livrable Fiverr/Upwork standard).

Critère	Freelance classique	JARVIS OS
Temps (demande → livrable)	~2 h de travail · livraison 1-2 j	36 secondes
Coût	25-60 \$	0 € (local, 0 token cloud)
Qualité	variable, JS souvent raté	1000/1000 lignes , 4 champs complets, CSV valide
Confidentialité	données chez le freelance	100 % on-premise

Synthèse — les chiffres qui comptent

- **99,6 %** de succès de routage sur **4 413 appels** LLM réels, tentatives moyennes **0,26**.
- **91,8 % du trafic LLM reste sur M1 local** (98,6 % local total), **< 1 % cloud gratuit, 0 token payant**.
- Réponse interactive médiane **~1,1 s** ; meilleur débit **51 tok/s** (cloud gratuit).
- Interprète vocal local : **WER 10,51 %**, **20,2× temps réel**, sur **~1 % d'un petaFLOP — 0 €** vs **1 440-2 500 \$/an** cloud.
- RAM **108 GB/s**, ZRAM **3,07:1**, CPU **14 560 ev/s** (16T) — machine saine sous charge.
- Livrable freelance standard : **36 s / 0 €** au lieu de 1-2 jours / 25-60 \$.

3. Hardware, BIOS & couche système

03 · Optimisations Hardware, BIOS & Couche Système — JARVIS OS (M1)

Données relevées en direct sur M1 le 2026-07-17. Toutes les valeurs proviennent de `/proc`, `/sys`, `nvidia-smi`, `zramctl`, `free`. “Linux de base” = comportement par défaut Ubuntu 22/24 LTS sans modification.

1. CPU — AMD Ryzen 7 5700X3D

Paramètre	Linux de base	JARVIS M1
Modèle	AMD Ryzen 7 5700X3D 8C/16T	idem
Cache L3	96 MiB V-Cache (3D)	96 MiB V-Cache
Fréquence max	4149 MHz (boost auto)	4149 MHz confirmée
Fréquence min	550 MHz (idle C-state)	550 MHz
Fréquence courante (relevée)	variable / schedutil	4026 MHz (governor performance)
Governor CPU	schedutil (Ubuntu)	performance — boost constant, pas de latence de remontée
Driver P-state	acpi-cpufreq	amd_pstate=active — interface native AMD (Collaborative P-State Control)
Preferred Core	désactivé	amd_pfc=1 — scheduler noyau redirige vers le cœur le plus rapide
BogoMIPS	—	5999
Virtualisation	AMD-V activé	AMD-V actif
Instructions SIMD	SSE4.2, AVX2, FMA, AES-NI, SHA	idem — accélération crypto/ML native

Impact mesuré (bench 2026-07-16) : 14 560 événements/s multi-thread, 1 802 ev/s single (charge concurrente ~7), rapport HT réel 8,08x (attendu pour SMT Zen3).

2. Paramètres BIOS / Boot Kernel

Paramètre cmdline	Valeur	Effet
<code>amd_pstate=active</code>	actif	Interface CPPC native AMD — P-states fins, latence plus basse que acpi-cpufreq
<code>amd_pfc=1</code>	actif	Preferred Core : boost prioritaire sur le cœur champion
<code>pcie_aspm=off</code>	désactivé	Suppression ASPM PCIe (économie d'énergie PCIe) — fix crash multi-GPU (hang modeset + watchdog reboot boucle sur ASMedia)
<code>mitigations=off</code>	actif	Désactivation Spectre/Meltdown/MDS — +5-15% CPU sur workloads syscall-intensifs
		THP activé globalement (2 MiB pages)

<code>transparent_hugepage=always</code>	always	— réduit la pression TLB sur gros workloads LLM
<code>loglevel=0</code>	0	Pas de bruit kernel sur console
<code>logo.nologo</code>	actif	Boot silencieux

Noyau actif : **Linux 6.8.0-117-generic** PREEMPT_DYNAMIC (Ubuntu SMP, 2026-05-05).

3. ZRAM — Swap compressé en RAM

Paramètre	Linux de base	JARVIS M1
ZRAM activé	non	oui — <code>/dev/zram0</code>
Taille brute	—	24 GiB
Algorithme	—	zstd (meilleur ratio, CPU moderne)
Threads	—	16 (= nb de vCPU)
Priorité swap	—	100 (priorité maximale, avant tout swap disque)
Ratio compression mesuré	—	3,07:1 (6,36 GiB données → 2,07 GiB compressés, relevé sous charge)
Swap physique (dm-0)	swap unique	16 GiB, priorité 10 (dernier recours)
Swappiness	60 (Ubuntu)	10 — préférence forte pour garder en RAM
Zero-pages ZRAM	—	97 236 pages (économie supplémentaire)

Capacité totale swap : 39 GiB (24 GiB ZRAM + 16 GiB dm-0). **État sous charge** : swap-in quasi nul, ZRAM sert de coussin élastique sans latence disque. Aucun OOM observé sur 46 GiB RAM totale.

4. RAM & Mémoire

Métrique	Valeur relevée
RAM totale	46 GiB (49 244 780 kB)
RAM disponible (relevé live)	18 GiB (19 703 020 kB)
RAM utilisée (live)	28 GiB (charge complète : LLM + Docker + OS)
Cache/tampon noyau	21 GiB
HugePages (statiques 2 MiB)	512 pages = 1 GiB réservé (496 libres, 16 en usage)
AnonHugePages (THP dynamiques)	11 315 200 kB ≈ 10,8 GiB alloués par THP=always
vfs_cache_pressure	50 (Ubuntu=100) — noyau conserve plus longtemps le cache VFS
dirty_ratio	20 (défaut Ubuntu)
dirty_background_ratio	5
dirty_writeback_centisecs	500

Débit RAM mesuré (sysbench) : 107 954 MiB/s séquentiel 8 threads. Plancher DRAM hors L3 (STREAM dataset > 96 MiB V-Cache) : ~15,4 GB/s — correspond aux 2 canaux DDR4.

5. I/O — Scheduler disque

Disque	Type	Scheduler	Linux de base
sda	HDD/SSD SATA	<code>none</code> (passthrough)	<code>mq-deadline</code>
sdb	HDD/SSD SATA	<code>none</code>	<code>mq-deadline</code>
sdc	HDD/SSD SATA	<code>none</code>	<code>mq-deadline</code>
sdd	HDD/SSD SATA	<code>none</code>	<code>mq-deadline</code>

`none` = pas d’ordonnancement supplémentaire dans le noyau, confiance au driver de bas niveau (optionnel moderne pour NVMe/SATA rapide). `mq-deadline` reste disponible dans la liste mais non sélectionné.

Bench disque mesuré : randread 4K QD1 = 8 730 IOPS (34,1 MiB/s) · randwrite 4K QD1 = 19 200 IOPS (74,9 MiB/s).

6. Hugepages & THP

Paramètre	Linux de base	JARVIS M1
<code>transparent_hugepage</code>	<code>madvise</code> (Ubuntu)	<code>always</code> (cmdline boot)
HugePages statiques (2 MiB)	0	512 pages (1 GiB) pré-réservé
AnonHugePages dynamiques	faible	10,8 GiB alloués (THP kernel actif)

THP `always` permet au noyau de mapper automatiquement les grandes zones mémoire en pages 2 MiB, réduisant la charge TLB pour les workloads LLM (modèles >4 GiB chargés en continu).

7. GPU — Topologie & Power Management

Index CUDA	Modèle	VRAM	Power Limit	Défaut	Rôle JARVIS
GPU 0	RTX 2060	12 288 MiB	150 W	184 W	Affichage GNOME — exclu des LLM
GPU 1	GTX 1660 SUPER	6 144 MiB	70 W	125 W	Inférence
GPU 2	GTX 1660 SUPER	6 144 MiB	70 W	125 W	Inférence — ventilo défaillant (0% fan, 74°C au relevé, jusqu’à 88°C en pic sous charge), surveiller
GPU 3	GTX 1660 SUPER	6 144 MiB	70 W	125 W	Inférence
GPU 4	RTX 3080	10 240 MiB GDDR6X	230 W	—	LLM principal (poids lourds)

VRAM totale disponible pour l’inférence (hors GPU0 affichage) : 28 672 MiB (28 GiB).

Règles de protection GPU actives

Règle	Raison	Fix appliqué
GPU écran (RTX 2060) exclu des LLM	Famine VRAM → hang modeset → watchdog reboot boucle	<code>disabledGpus:[1]</code> LM Studio + <code>CUDA_VISIBLE_DEVICES</code> remapping
<code>pcie_aspm=off</code> en cmdline	Crash multi-GPU sur bus ASMedia PCIe (ASPM économie d’énergie corrupteur)	Paramètre boot permanent
Module NVIDIA open + GSP désactivé	Multi-crash GPU confirmé sur driver	Suppression <code>nvidia-gsp.conf</code>

	open + GSP activé	
Power limit RTX 2060 : 150 W (vs 184 W défaut)	Réduction chaleur GPU affichage, VRAM préservée pour OS	nvidia-smi -pl 150
GPU 2 (1660S) : ventilateur défaillant	Fan 0% — 74°C au relevé courant, pic mesuré ~88°C sous charge intensive — risque thermique	Exclu des benchmarks intensifs

Températures & VRAM live (relevé 2026-07-17)

GPU	Temp	Fan	VRAM utilisée
GPU 0 RTX 2060 (écran)	54°C	46%	4 114 / 12 288 MiB
GPU 1 GTX 1660S	44°C	0%	972 / 6 144 MiB
GPU 2 GTX 1660S	74°C	0% (défaillant)	3 231 / 6 144 MiB
GPU 3 GTX 1660S	44°C	0%	2 096 / 6 144 MiB
GPU 4 RTX 3080	72°C	46%	8 745 / 10 240 MiB

8. Résumé des optimisations (Linux de base → JARVIS)

Couche	Linux de base	JARVIS M1	Gain
CPU governor	schedutil	performance	Latence remontée fréquence supprimée
AMD P-state	acpi-cpufreq	amd_pstate=active	P-states fins natifs AMD
Preferred Core	off	amd_prefcore=1	Cœur champion priorisé
Mitigations CPU	on	off	+5-15% sur syscall-bound
PCIe ASPM	on	off	Fix crash GPU multi-reboot
THP	madvise	always	~10,8 GiB en huge pages, TLB réduit
HugePages statiques	0	512 × 2 MiB = 1 GiB	Réservation mémoire prévisible
ZRAM	absent	24 GiB zstd prio=100	Swap sans latence disque, ratio 3:1
Swappiness	60	10	RAM préservée, swap dernier recours
vfs_cache_pressure	100	50	Cache VFS retenu plus longtemps
I/O scheduler	mq-deadline	none	Passthrough driver, latence minimale
Driver NVIDIA	open + GSP	GSP désactivé	Fix crash multi-GPU
LLM sur GPU écran	possible	interdit (règle config)	Famine VRAM → reboot évité

4. Widget bureau « Ce que le système fait »

Le widget bureau « Ce que le système fait » — présentation & utilisation

Vision

Un **tableau de bord de bureau temps réel, zéro token**, qui répond à une seule question : « *qu'est-ce que JARVIS est en train de faire, là, maintenant ?* ». Il ne consomme aucun crédit LLM : il **lit directement** l'état de la machine (bases SQLite, logs, `nvidia-smi`, `systemctl`) et l'affiche en continu.

Architecture technique

Composant	Rôle	Détail
Backend <code>bin/jarvis-planning-widget.py</code>	Serveur HTTP stdlib, port 8899	<code>sert /</code> (dashboard HTML auto-refresh) + <code>/data</code> (JSON) + <code>POST /trigger</code>
Fenêtre <code>bin/jarvis-widget-desktop.py</code>	Vue WebKitGTK sans bordure, <code>keep_below</code>	pompe le rafraîchissement depuis Python (Glib)
Supervision	2 services systemd <code>--user</code>	<code>Restart=always</code> → relance < 5 s après un kill/crash

Le rafraîchissement est **piloté côté Python** et non par le JavaScript de la page : WebKit gèle `setInterval` pour les fenêtres en arrière-plan (`keep_below`), donc `Glib.timeout_add_seconds` pompe `tick()` / `cdTick()` pour garder le compte à rebours et les données vivants en permanence.

Les 5 panneaux (lecture seule, 0-token)

Panneau	Source	Ce qu'il montre
File de tâches	<code>jarvis_master.db</code>	en file / faites / analysées
Routage LLM	<code>llm_cascade_log.jsonl</code>	canaux de sortie, % fallback (fenêtre courte n=60)
Workflows n8n	<code>~/.n8n/database.sqlite</code>	workflows auto-déclenchés, ok/ko 24 h
GPU thermique	<code>nvidia-smi</code>	température, ventilateur, charge des 5 GPU
Timers planifiés	<code>systemctl --user list-timers</code>	prochaines échéances + compte à rebours

Compte à rebours & auto-déclencheur

En haut du widget, un **compte à rebours** indique le **prochain auto-déclenchement** (ex. `jarvis-hub-healthcheck` dans 3 s). C'est la preuve visuelle que le système s'anime tout seul : timers systemd → actions → mise à jour du dashboard.

Boutons d'action (POST /trigger, durci)

Le widget n'est pas que passif : une **barre d'actions** déclenche des services via `POST /trigger`, protégé par une **liste blanche** + anti-CSRF / anti-DNS-rebind (seule la page servie depuis `127.0.0.1:<port>` peut déclencher ; toute `Origin` cross-site ou `Host` non-loopback → **403**).

Bouton	Action déclenchée
▶ exécuter file	traite la file de tâches en attente
▶ régénérer todo	reconstruit la todoliste dynamique
▶ remplir biblio	relance le daemon Bibliothèque Vivante (0-token)
▶ web-cascade	construction web réelle (anti-hallucination)

▶ vectoriser	indexation vectorielle des fiches
✉ envoyer lot (40)	envoi du lot de prospection courant
📷 snapshot campagne	capture l'état d'une campagne
📁 secteurs demande	agrège la demande par secteur

Indicateurs clés (KPI, exemple live)

39 en file · 181 faites · 38 analysées · 200 fiches produites · 44 timers actifs — GPU 44–66 °C · routage fallback 2 % (sain).

Utilisation au quotidien

```
## État des deux services (prod)
systemctl --user status jarvis-planning-widget.service # backend :8899
systemctl --user status jarvis-widget-desktop.service # fenêtre bureau

## Après édition du backend Python
systemctl --user restart jarvis-planning-widget.service

## Après édition du HTML/JS du dashboard (la fenêtre cache l'ancien HTML)
systemctl --user restart jarvis-widget-desktop.service

## Consulter les données brutes en JSON
curl -s http://127.0.0.1:8899/data | python3 -m json.tool
```

Débloquer un flapping (après plusieurs restarts rapides, start-limit-hit) :

```
systemctl --user reset-failed jarvis-planning-widget.service
systemctl --user start jarvis-planning-widget.service
```

Robustesse — pourquoi il ne « disparaît » plus

Auparavant la fenêtre était un simple autostart GNOME **sans superviseur** : un kill la faisait disparaître jusqu’au prochain login. Elle est désormais un **service systemd supervisé** (Restart=always) ; l'autostart a été neutralisé (.desktop.disabled) pour éviter le double lancement. Les panneaux se **dégradent proprement** (vides) si une source manque (nvidia-smi absent, base n8n verrouillée) — jamais de crash.

Démonstration vidéo narrée

Une capture d’écran vidéo **narrée** (voix off française, edge-tts) accompagne ce pack : demo/demo-widget-jarvis-*.mp4 — elle montre le widget en fonctionnement réel avec le compte à rebours vivant, les KPI, le thermique GPU et les tâches planifiées.

5. Écosystème d’agents

Écosystème d’agents — le décompte réel

Le chiffre « **mille agents** » n’est pas un slogan : c’est l’**arrondi de 961 agents réellement indexés**. Voici la preuve, source par source, avec trois décomptes concordants et vérifiables.

Décompte 1 — Index cowork (le « 1000 »)

`~/claude/plugins/jarvis-cowork/agent-index.json` : **961 agents** indexés (scripts spécialisés, chacun avec `source / path / desc`).

Source	Agents
services	848
infra-scripts	82
jarvis-core	28
workers	3
Total	961

Décompte 2 — Comptage entreprise toutes couches (1 354)

`jarvis-core/docs/agent-entreprise/comptage-final.md` (source vérifiée 2026-06-18) — consolide **toutes** les couches d’agents :

Couche	Source	Nb réel
Index cowork	<code>jarvis-cowork/agent-index.json</code>	961
Agents Claude natifs (<code>.md</code> stricts)	<code>.claude/agents/*.md</code>	58
Agents plugins (jarvis-os / turbo / cowork)	<code>.claude/plugins/</code>	71
Skills-agents	<code>.claude/skills/</code> (SKILL.md)	169
Actions OpenClaw	API M5:8400	95
TOTAL	—	1 354

Décompte 3 — Inventaire OMEGA (1 435, 11 catégories)

`inventaire-complet-1435-agents.md` (horodaté 2026-03-29) — le décompte le plus large, incluant les Legions et les chaînes Domino :

#	Catégorie	Quantité	Source
1	Legions	600 (10×60)	<code>openclaw-master.py</code>
2	Cowork Scripts	579	<code>modules/cowork/</code>
3	Skills Gemini CLI	85	<code>~/gemini/skills/</code>
4	Domino Chains	71	<code>core/domino/chains.d/</code>
5	Agents SQL Registry	38	<code>jarvis_orchestrator.db</code>
6	CLIs système	18	<code>/usr/local/bin/</code>
7	Agents Core Python	14	<code>core/agents/*.py</code>
8	Agent Modules	13	<code>core/agent_modules/*.py</code>

9	OpenClaw Skills	11 (46 cmds)	openclaw/skills/
10	Omega Master Agents	6	~/.claude/agents/ (sous-ensemble des 58 .md)
11	(divers)	—	—
TOTAL		1 435	

Synthèse — comment présenter le chiffre

Formulation	Chiffre	Quand l'utiliser
« près de mille agents indexés »	961	affirmation forte et vérifiable en une commande
« plus de 1 350 agents toutes couches »	1 354	décompte entreprise consolidé (2026-06-18)
« jusqu'à 1 435 composants actifs »	1 435	inventaire OMEGA complet (Legions incluses)

Un agent, ici, = un composant exécutable spécialisé (script de service, skill, chaîne Domino, action OpenClaw, ou agent Claude natif), routé par le hub et pilotable par la voix. Les **129 agents Claude Code** natifs n'en sont que la façade la plus visible.

Réconciliation du « 129 ». Ce chiffre = agents Claude Code chargés en session sur M1 (agents natifs + plugins jarvis-os / turbo / cowork). Il ne faut pas le confondre avec : les **58** fichiers `.md` natifs stricts de `~/.claude/agents/` (retenus tels quels dans le total entreprise 1 354, décompte 2 ci-dessus), ni avec les **6** « Omega Master Agents » de `~/.claude/agents/` comptés dans l'inventaire OMEGA (décompte 3, catégorie 10). Trois périmètres emboîtés, pas trois valeurs concurrentes : $6 < 58 < 129$. Les totaux headline restent **961** indexés · **1 354** toutes couches · **1 435** OMEGA.

Infrastructure de support : **26 containers Docker** actifs répartis M1/M2/M5, **2 533 scripts Python** (jarvis-linux) + **618 scripts cowork**.

6. Démonstration vidéo

Une capture d'écran **vidéo narrée** (voix off française) accompagne ce dossier :

```
demo/demo-widget-jarvis-20260717-003517.mp4  
demo/demo-widget-jarvis-20260717-003645.mp4  
demo/demo-widget-jarvis-20260717-005750.mp4
```

Elle montre le widget en fonctionnement réel — compte à rebours vivant, KPI, thermique GPU, tâches planifiées — avec une narration décrivant JARVIS OS, ses performances et ses différences face à un Linux de base.